

Container, Basics und Konzepte



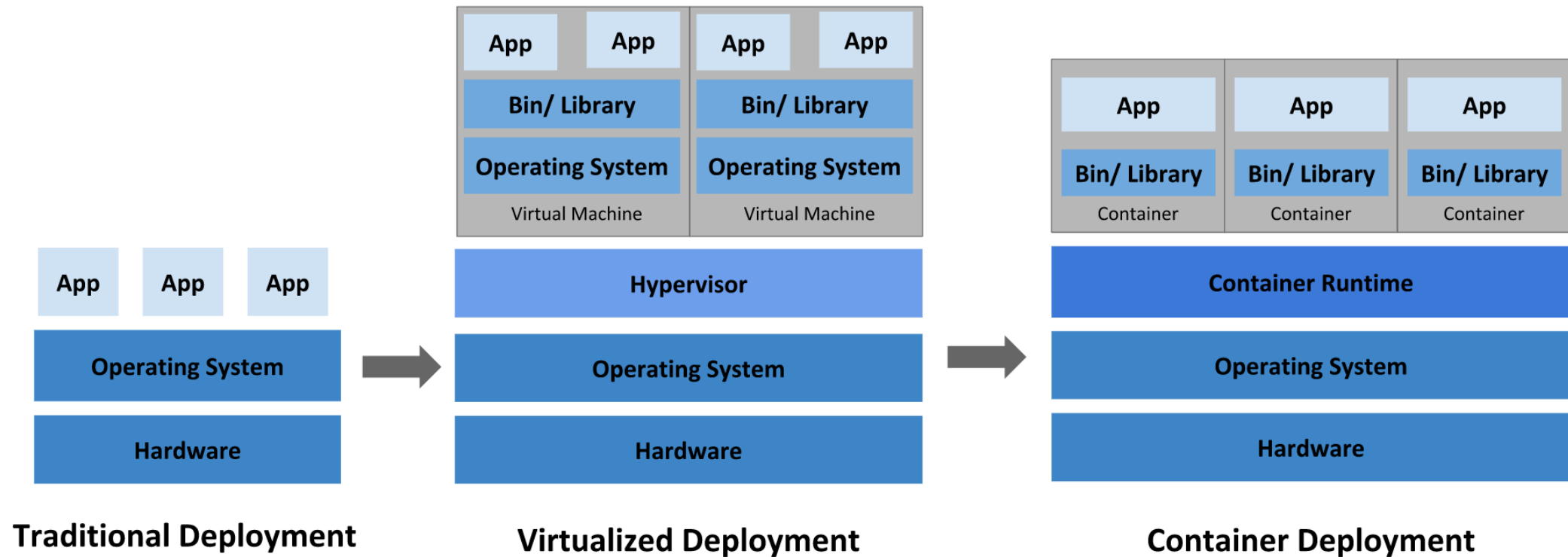
Lernziele

- Sie haben einen Einblick in die Container Basics

Zeitlicher Ablauf

- Von der traditionellen Informatik, zu Virtuellen Maschinen und Containern
- Container-Geschichte
- Wer verwendet was?
- Container Standards: Open Container Initiative
- Übung: Container Umgebungen
- Reflexion
- Lernzielkontrolle

Von der traditionellen Informatik, zu Virtuellen Maschinen und Containern (1)



Von der traditionellen Informatik, zu Virtuellen Maschinen und Containern (2)

Virtuelle Maschinen

- Hypervisor und Guest OS
- Applikation und ihre Abhängigen z.B. Libraries / DLLs in VM
- **Schwergewichtig** (Platzbedarf: GB)
- ISO-Image (CD-Rom) und **Manuelle** Installation von Applikationen oder **viele Installations-Script** und Produkte z.B. Cloud-init, Ansible, Chef, Puppet, Vagrant etc.

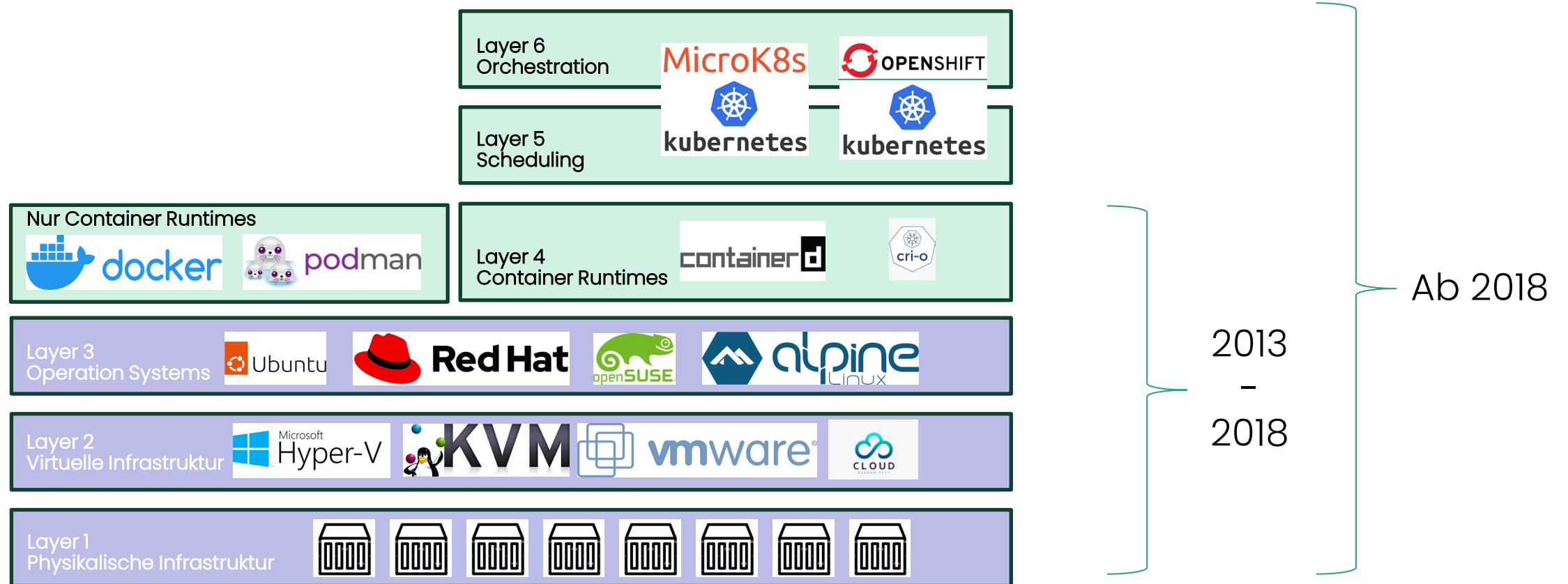
Container

- Container Runtime
- Applikation (Microservice) und ihre Abhängigen z.B. Libraries / DLLs **im Container Image**.
- **Leichtgewichtig** (Platzbedarf: KB – MB)
- (Automatische) Bereitstellung der Applikationen (Microservices) als Container Images. Paketierte mittels einer **einheitlichen** Beschreibungsdatei (Dockerfile), abgelegt in [Registries](#)

Container – Geschichte

- Container-Technologien haben ihre Wurzeln in den 1970er-Jahren. **1979** führte UNIX mit *chroot* erstmals Dateisystem-Isolation ein.
- **1998** folgten *FreeBSD Jails*, die auch Prozesse isolierten.
- **2001** kamen *Solaris Zones* sowie *Virtuozzo* (später als *OpenVZ*) hinzu, die Container auf Linux brachten.
- Google entwickelte anschliessend *cgroups* zur Ressourcenbegrenzung und legte damit die Basis für moderne Container.
- Das **LXC-Projekt (2008)** vereinte *cgroups*, *Namespaces* und *chroot* zu einer vollständigen Containerlösung
- Mit **Docker (2013)** wurde Containerisierung schliesslich massentauglich: einfach, portabel und effizient.
- Heute steht **Kubernetes** für die Orchestrierung grosser Containerlandschaften, während reine Docker-Umgebungen zunehmend an Bedeutung verlieren.

Container Geschichte: als vereinfachte Layer



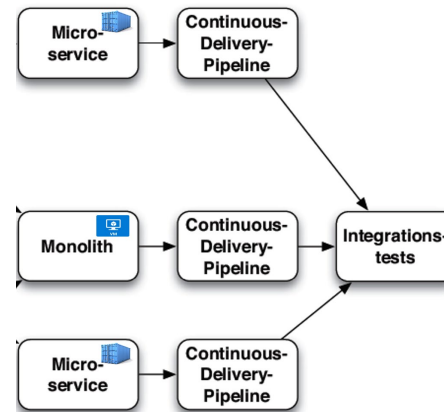
Die (vereinfachten) »Layer« der Container-Welt

Wer verwendet was? (Trend)

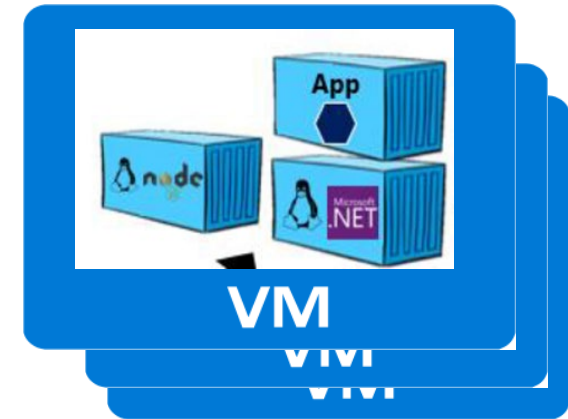
Entwicklung (Dev)



Tests/Integration/Paketierung



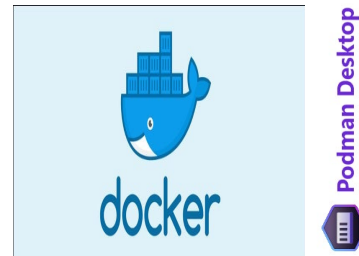
Betrieb (Ops)



Lokal auf dem Entwickler-NB



Build



Integrationstests

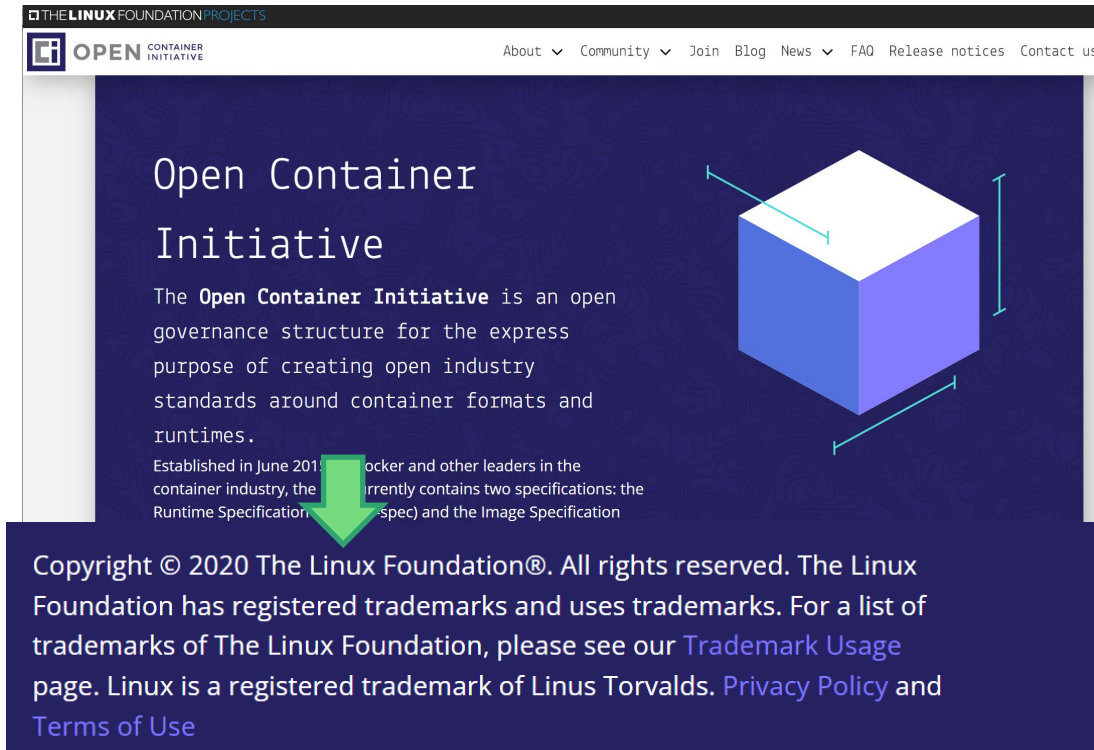


Operating



Anmerkung: Erfahrung des Kursleiters

Standard: Open Container Initiative



The screenshot shows the Open Container Initiative website. At the top, there's a navigation bar with links: About, Community, Join, Blog, News, FAQ, Release notices, and Contact us. The main heading is "Open Container Initiative". Below it, a paragraph states: "The Open Container Initiative is an open governance structure for the express purpose of creating open industry standards around container formats and runtimes." To the right of this text is a 3D cube graphic with blue and white faces and red dimension lines. Below the paragraph, it says: "Established in June 2015, Docker and other leaders in the container industry, the initiative currently contains two specifications: the Runtime Specification (spec) and the Image Specification". A green arrow points from this text down to a footer area. The footer contains copyright information: "Copyright © 2020 The Linux Foundation®. All rights reserved. The Linux Foundation has registered trademarks and uses trademarks. For a list of trademarks of The Linux Foundation, please see our Trademark Usage page. Linux is a registered trademark of Linus Torvalds. Privacy Policy and Terms of Use".

- Die [Open Container Initiative](#) (OCI) ist ein unter dem Dach der Linux Foundation gestartetes Projekt, um industrieweit gemeinsame Standards für die Container-Virtualisierung zu schaffen.
- Diese Initiative nutzt eine offene Governance-Struktur und wurde als Open Container Project auf der DockerCon im Juni 2015 vorgestellt. Später wurde sie zur Open Container Initiative umbenannt.
- Spezifikationen:
 - ◇ "[runtime-spec](#)" – Laufzeitverhalten (Runtime)
 - ◇ "[image-spec](#)" – Imageformat
 - ◇ Weitere in Entwicklung, z.B. Registry (oci://)

Jupyter Notebooks – Übungsumgebung

Run

Einzelne Zeile ausführen. * Zeile wird ausgeführt <1-xx> Zeile wurde ausgeführt Zeilen können mehrfach ausgeführt werden	
+	Zeile hinzufügen
!<Befehl>	Ausführen eines Kommandozeilenbefehls (CLI)
%%bash	Ausführen mehrere Befehle
Letzte Zeile dient zum Aufräumen, nicht ausführen	

```
In [ ]: ! kubectl apply -f https://raw.githubusercontent.com/mc-b/misegr/master/ewolff/ms-kubernetes/postgres.yaml
! kubectl apply -f https://raw.githubusercontent.com/mc-b/misegr/master/ewolff/ms-kubernetes/apache.yaml
! kubectl apply -f https://raw.githubusercontent.com/mc-b/misegr/master/ewolff/ms-kubernetes/catalog.yaml
! kubectl apply -f https://raw.githubusercontent.com/mc-b/misegr/master/ewolff/ms-kubernetes/order.yaml
! kubectl rollout restart deployment.apps/apache
! kubectl rollout restart deployment.apps/catalog
! kubectl rollout restart deployment.apps/order
```

Alternative: Linux prompt in der Virtuellen Maschine

```
vagrant@master:~$ kubectl apply -f https://raw.githubusercontent.com/mc-b/misegr/master/ewolff/ms-kubernetes/apache.yaml
service/apache unchanged
deployment.apps/apache unchanged
vagrant@master:~$ kubectl apply -f https://raw.githubusercontent.com/mc-b/misegr/master/ewolff/ms-kubernetes/catalog.yaml
service/catalog created
deployment.apps/catalog created
vagrant@master:~$
```

- Ein [Jupyter Notebook](#) ist eine Open-Source-Webanwendung, mit der Sie wiederholende **Abläufe erstellen** können. Die **Live-Code** (**hier Linux Befehle**), Gleichungen, Visualisierungen und Beschreibungen enthalten können.
- Mittels werden die Zeilen, nach und nach an die VM weitergeleitet.
- Durch anklicken einer Zeile, kann diese auch mehrmals ausgeführt werden.
- Alternativ können die Zeilen die mit "!" oder unter "%%bash" stehen auch in der VMs ausgeführt werden.
- Jupyter Notebooks: **nach dem Kurs**
 - ◊ <https://github.com/mc-b/duk> - Wechsel in Verzeichnis /data/jupyter

Übung 01-1: Container Standards

Jupyter 01-1-Container-Standards Last Checkpoint: 2 minutes ago

File Edit View Run Kernel Settings Help Truste

JupyterLab Python 3 (ipykernel)

Übung: Container Standards und Container Umgebungen

Die [Open Container Initiative \(OCI\)](#) ist eine gemeinnützige Organisation, die 2015 unter dem Dach der Linux Foundation gegründet wurde, um offene Industriestandards für Container-Formate und -Runtimes zu definieren und zu fördern.

Diese Standards sind entscheidend für die Interoperabilität und Kompatibilität verschiedener Container-Technologien, wie Docker, Podman und Kubernetes.

Durch das Starten desselben Containers in Docker, Podman und Kubernetes demonstrieren wir die Interoperabilität und Kompatibilität dieser Technologien.

Wir beginnen mit Docker:

```
[ ]: ! docker run --name webshop-docker -d -p 8080:8080 registry.gitlab.com/ch-mc-b/autoshop/shop:2.0.0
! echo "http://$(cat ~/work/server-ip):8080"
```

Gehen weiter zu Podman:

```
[ ]: ! podman run --name webshop-podman -d -p 9090:8080 registry.gitlab.com/ch-mc-b/autoshop/shop:2.0.0
! echo "http://$(cat ~/work/server-ip):9090"
```

Und schliessen mit Kubernetes ab:

```
[ ]: %bash
kubectl run webshop-kubernetes --image registry.gitlab.com/ch-mc-b/autoshop/shop:2.0.0 --image-pull-policy="IfNotPresent" --restart=Never
kubectl expose pod/webshop-kubernetes --type="LoadBalancer" --port 8080
echo "http://$(cat ~/work/server-ip):$(kubectl get service webshop-kubernetes -o=jsonpath='{.spec.ports[0].nodePort}')"
```

- **Ergebnis:** Webshop verpackt in einem Container. Läuft ohne Änderungen in drei verschiedenen Container Umgebungen.
- **Erkenntnisse:** Es existieren zwar verschiedene Container-Umgebungen, doch dank der Open Container Initiative (OCI) sind die erstellten Images plattformübergreifend kompatibel.

Auto Shop on webshop-kubernetes

Customer

Catalog

Order



Reflexion

- Container sind ein altes Konzept
- Sie sind aus dem Wunsch entstanden, Applikationen und/oder Services voneinander zu isolieren ohne die schwere Gewichtigkeit von Virtuellen Maschinen
- Bei den Container Umgebungen hat Kubernetes die grösste Community
- **Container (Runtime und Imageformat) sind Standardisiert!**

Lernzielkontrolle

- Sie haben einen Einblick in die Container Basics.

Übung: Container Umgebungen

Vergleicht Docker (Produkt und eine Firma) mit den Zertifizierten Kubernetes Umgebungen

<https://www.docker.com/company/>

- Docker Desktop – **für Entwickler** (für grosse Firmen kostenpflichtig)
- Docker EE – **verkauft an [Mirantis](#)**
- **Kubernetes is [deprecating Docker](#)** (Ende Unterstützung **30.04.22**)

Docker, Inc.

Unternehmen



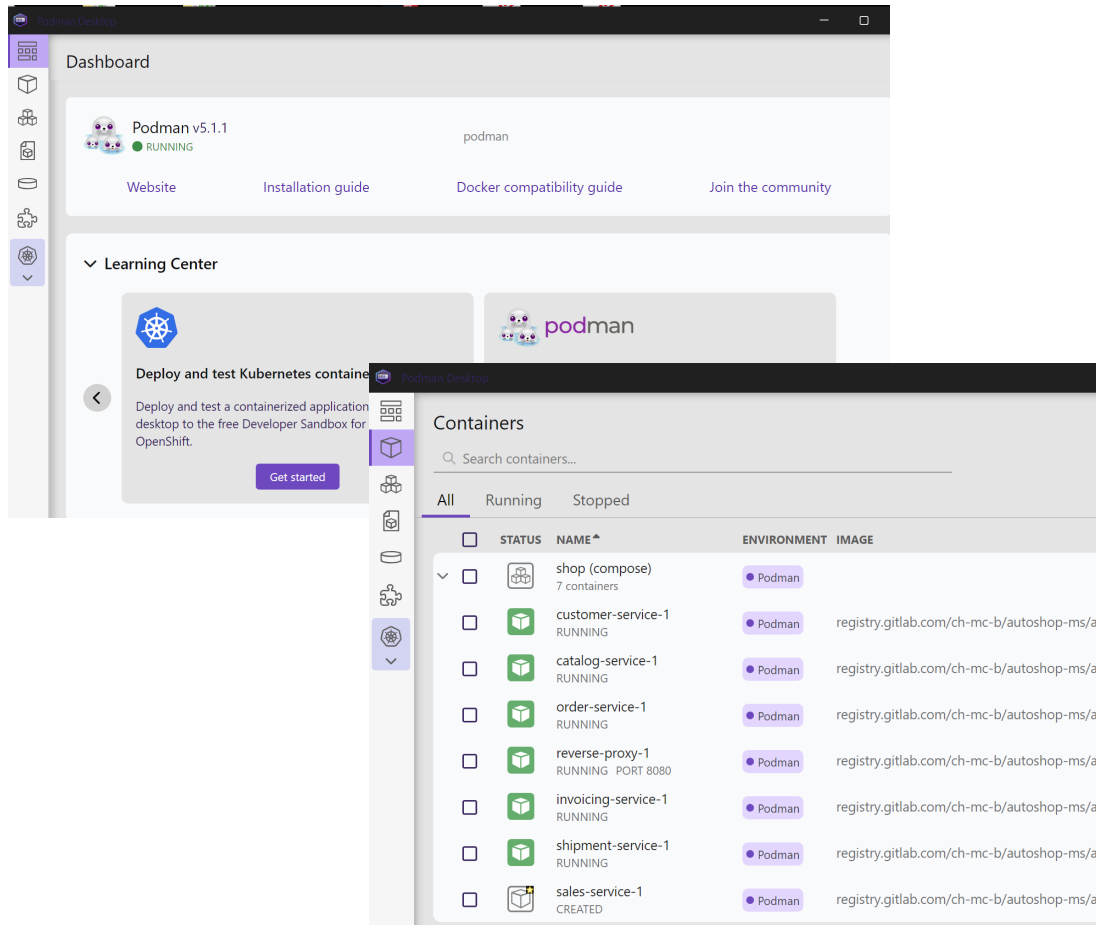
Aus dem Englischen übersetzt - Docker, Inc. ist ein amerikanisches Technologieunternehmen, das Docker Hub und Docker Desktop entwickelt, Entwicklerproduktivitätstools, die auf Docker basieren, einem Open-Source-Projekt, das die Bereitstellung von Code in Software-Containern automatisiert. [Wikipedia \(Englisch\)](#)

[Ursprüngliche Beschreibung aufrufen](#) ▼

Gründung: 2008

Hauptsitz: [Palo Alto, Kalifornien, Vereinigte Staaten](#)

Docker Desktop Alternative: Podman Desktop



- Container verwalten
 - ◇ Container auflisten, suchen, prüfen, verbinden, ausführen und stoppen
- Images erstellen, pull und push
 - ◇ Erstellen Sie Container Images aus dem Tool und stellen diesen in Registries ab
- Learning Center
- Kompatibel mit Docker Desktop-Erweiterungen (z.B. Deploy to OpenShift)
- <https://podman-desktop.io/>